Multi-Source Candidate Data Transformer — Design Doc

Author: Neha | Email: neha.contact295@gmail.com

1. Pipeline

`extract → normalize → merge → confidence → provenance → project (config) → validate`

Each source is parsed independently into a common intermediate format (raw fields + source tag). Normalize standardizes formats per field. Merge deduplicates candidates across sources using email as the primary match key (fallback: normalized full name). Confidence and provenance are computed per field during merge. Project reshapes the canonical record per the runtime config. Validate checks the final output against the requested schema before returning.

2. Output Schema

Using the assignment's default schema as-is: candidate_id, full_name, emails[], phones[] (E.164), location {city, region, country (ISO-3166 alpha-2)}, links {linkedin, github, portfolio, other[]}, headline, years_experience, skills [{name, confidence, sources[]}], experience [{company, title, start, end, summary}], education [{institution, degree, field, end_year}], provenance [{field, source, method}], overall_confidence.

Normalization: phones → E.164 (+91 default if no country code); dates → YYYY-MM; skills → lowercased, deduped, canonicalized (e.g. “JS” → “javascript”).

3. Merge & Conflict Resolution

Match key: normalized email, fallback normalized full name. **Source priority** for scalar conflicts (name, phone): CSV > ATS > GitHub, since recruiter-entered structured data is treated as more reliable than public/scraped data. **List fields** (skills, experience) are unioned across sources rather than overwritten.

Field confidence = (# sources agreeing on chosen value) / (# sources providing that field). **Overall confidence** = mean of all populated field confidences.

4. Runtime Configuration Handling

A separate projection layer reads a JSON config and reshapes the canonical record — the canonical record is never mutated. Supports: field subset selection, “from” path remapping (dot + array notation), per-field normalize overrides, include_confidence / include_provenance toggles, and on_missing behavior (null / omit / raise error). Output is validated against the requested config's field list before returning.

5. Edge Cases Handled

- Missing email on a source → falls back to name-based matching, lower match confidence
- Malformed/missing phone → normalization fails gracefully, field becomes null, logged as warning
- GitHub API 404 or rate-limited → that source marked failed, pipeline continues with remaining sources
- Inconsistent name spelling across sources → matched via email, name field picked by source priority
- Missing/garbage source file entirely → skipped with a warning, pipeline doesn't crash

6. Descoped (due to time)

LinkedIn parsing, resume PDF/DOCX parsing, recruiter free-text notes, and an automated test suite were intentionally left out as honest scope cuts under time pressure. The core engine (extract / normalize / merge / confidence / provenance / project / validate) and the CLI were fully implemented and prioritized, per the assignment's stated evaluation criteria.